
PL & Partners

Internal Workspace Architecture

System Architecture and Technical Design Report

Prepared by:
Kevin Kluka

August 2026

Contents

1	Introduction	3
1.1	Context and Background	3
1.2	Problem Statement and Project Objectives	3
2	Analysis	3
2.1	Architectural Stack Ecosystems	3
2.2	TypeScript Stack (Node.js / NestJS)	4
2.3	Frontend Tier: Angular vs. React	4
2.4	Application Tier: C# (.NET) vs. Java (Spring Boot)	4
2.5	Data Tier: PostgreSQL/MySQL/MongoDB/SQLite	5
3	Deployment Topology and Pricing Analysis	5
3.1	Topology 1: The Modern Composable Stack (Selected)	5
3.2	Topology 2: Amazon Web Services (AWS)	6
3.3	Topology 3: Microsoft Azure	6
3.4	Topology 4: Cloudflare (The Edge Architecture)	7
3.5	Conclusion	7
3.5.1	Presentation Layer: Vercel	7
3.5.2	Application Processing Layer: Railway (ASP.NET Core API)	7
3.5.3	Data Persistence Layer: Supabase (Managed PostgreSQL)	8
4	Deployment Strategy and CI/CD	8
4.1	Continuous Integration and Continuous Deployment (CI/CD)	8
4.1.1	Frontend CI/CD via Vercel	8
4.1.2	Application CI/CD via Railway	8
4.1.3	Data Tier CI/CD via Supabase	9
4.1.4	Automated Testing and CI via GitHub Actions	9
5	Security and Access Control	9
5.1	Application-Level Authorization	9
5.2	Data-Level Authorization (Defense in Depth)	10
5.3	The Transport Layer handles encryption (HTTPS)	10
5.4	Cryptographic Controls	10
5.5	Compliance and ISO/IEC 27001 Alignment	10
5.6	Risk Management Framework (ISO 31000 Alignment)	11
6	System Architecture Topology	11
7	Frontend Architecture and Navigation	12
7.1	Global Pages (Cross-Client Navigation)	12
7.2	Workspace-Specific Pages (Tenant Context)	13
8	Implementation and Local Tooling	13

8.1	Local Development Environment	13
8.2	Project Directory Structure	14
8.2.1	Frontend (Angular)	14
8.2.2	Backend (ASP.NET Core)	14
8.3	Database Schema and Entity Relationships	15

1 Introduction

1.1 Context and Background

Modern internal agency tools rely heavily on multi-tenant architectures to ensure data isolation, maintainability, and horizontal scalability. To support consultants working across various client accounts, the system must securely divide environments into distinct workspaces while utilizing a shared underlying infrastructure. A standard 3-tier architecture divides the system into three logical and physical layers:

- **Presentation tier** (user interface and dashboard rendering)
- **Application tier** (business logic, role-based access control, and batch data ingestion)
- **Data tier** (persistence storage for multi-platform advertising metrics).

Designing such a system requires careful selection of technologies across each tier to guarantee performance for analytical queries, security between client tenants, and developer productivity.

1.2 Problem Statement and Project Objectives

Building a reliable internal platform for PL & Partners requires a robust infrastructure capable of handling role-based access control (RBAC) for approximately 30 consultants, dynamic dashboard rendering, and nightly batch ingestion of advertising data from APIs such as Meta, Google, and TikTok.

The primary objective of this paper is to analyze candidate tech stacks across the presentation, application, and data tiers, and to propose a scalable, production-ready system design optimized for a Minimum Viable Product (MVP) of 5 client workspaces, with a seamless architectural runway to scale up to 100 workspaces.

2 Analysis

2.1 Architectural Stack Ecosystems

When evaluating full-stack ecosystems for a multi-tenant enterprise application, three primary candidates emerge:

1. **The .NET Ecosystem:** A strongly typed, highly optimized environment featuring ASP.NET Core for backend logic and Entity Framework Core for data access. It emphasizes performance, strong object-oriented design, and is highly efficient for running heavy ETL (Extract, Transform, Load) batch jobs.
2. **The Java Enterprise Stack:** Centered around Spring Boot and Hibernate, this ecosystem represents a battle-tested paradigm for enterprise systems. It provides extensive security frameworks but often requires higher initial boilerplate and memory overhead.
3. **The JavaScript/TypeScript Stack:** Combining Node.js/Express or NestJS with React or Vue, this paradigm minimizes context switching by utilizing TypeScript across all layers, prioritizing rapid development velocity.

2.2 TypeScript Stack (Node.js / NestJS)

While a unified TypeScript stack (using Node.js or NestJS on the backend alongside Angular on the frontend) offers the advantage of a single programming language across the entire codebase, it introduces key trade-offs for this specific case study:

- **CPU-Intensive ETL:** Node.js relies on a single-threaded event loop that risks blocking during heavy data transformations. Conversely, C# (.NET) natively excels at multi-threaded processing, making it significantly more efficient for aggregating years of historical ad metrics.
- **Memory Efficiency:** Parsing massive JSON payloads from advertising APIs often causes severe memory spikes in Node.js. ASP.NET Core mitigates this by utilizing high-performance, low-allocation memory streaming.
- **Background Job Management:** Running scheduled nightly batch tasks in Node.js typically requires deploying external infrastructure (e.g., Redis queues). ASP.NET Core simplifies this architectural footprint by offering robust, native background processing features out of the box.

2.3 Frontend Tier: Angular vs. React

For an internal tool centered around persistent navigation (a persistent left-hand menu) and complex data visualization (KPI charts, pacing tables), Angular provides a cohesive architectural foundation. Unlike React, which functions primarily as a view library requiring external dependencies for routing and state management, Angular operates as a fully featured framework.

Key advantages of Angular for this workspace design include:

- **Built-in Dependency Injection:** Encapsulates workspace state logic into modular services, making it easy to seamlessly switch data context when a consultant navigates between different client workspaces.
- **Native TypeScript Integration:** Mandates strict typing across template bindings, components, and API responses, reducing runtime errors when parsing complex ad metrics.
- **Modern Reactive Primitives:** Recent additions such as Standalone Components and Signals streamline local state reactivity for updating heavy analytical dashboards efficiently.

2.4 Application Tier: C# (.NET) vs. Java (Spring Boot)

Both C# and Java offer enterprise-grade capabilities for backend RESTful APIs, yet C# (.NET) provides distinct advantages for handling the nightly batch ingestion of ad data:

- **Language Expressiveness:** C# incorporates modern language features—such as Language Integrated Query (LINQ)—which vastly simplifies the filtering, grouping, and transforming of massive datasets retrieved from third-party advertising APIs.
- **Shared Paradigm with TypeScript:** Because C# and TypeScript share architectural principles, transitioning between Angular frontend services and C# backend controllers minimizes cognitive friction.
- **Execution Performance:** ASP.NET Core consistently demonstrates lower memory footprints

and faster startup times compared to conventional JVM environments, making it highly efficient for containerized background cron jobs.

2.5 Data Tier: PostgreSQL/MySQL/MongoDB/SQLite

Evaluating data persistence models requires balancing multi-tenant relational integrity against the flexibility needed to store varied payloads from Meta, Google, and TikTok:

- **PostgreSQL (Relational):** The optimal choice for multi-tenant systems. With Row-Level Security (RLS) and native JSONB support, PostgreSQL combines strict ACID compliance with the capability to store unstructured, multi-year ad metrics efficiently while securely isolating client data.
- **MySQL (Relational):** A highly reliable relational database optimized for read-heavy workloads, though lacking some of PostgreSQL's advanced JSON querying, custom indexing features, and native multi-tenant RLS capabilities necessary for varied advertising API payloads.
- **MongoDB (Non-Relational):** A document store that excels in handling dynamic schemas. However, aggregating complex diminishing return analyses across multi-year data is often less performant than native SQL joins and window functions.
- **SQLite (Embedded Relational):** An ultra-lightweight, zero-configuration embedded database. While excellent for local testing or single-user applications, its file-locking mechanism creates write-concurrency bottlenecks during nightly batch ETL ingestions, and it lacks native network client-server capabilities and Row-Level Security required for multi-tenant cloud operations.

3 Deployment Topology and Pricing Analysis

Selecting the optimal deployment topology requires balancing monthly operational costs against developer experience (DX), infrastructure complexity, and architectural compatibility. For an internal B2B tool serving 30 to 100 consultants, bandwidth costs are negligible; instead, the primary cost drivers are the persistent database and the always-on compute required for the backend API and nightly ETL background jobs.

The table below outlines four distinct deployment topologies, analyzing their component breakdown, architectural fit for an ASP.NET Core application, and estimated monthly costs (based on 2026 pricing models).

3.1 Topology 1: The Modern Composable Stack (Selected)

Stack: Vercel (Frontend) + Railway (Backend) + Supabase (Database).

Estimated Cost: ~\$65/month.

- **Vercel (\$20/mo):** A single Pro seat covers the frontend hosting, preview environments, and SSL configurations. Bandwidth usage for 30 consultants will comfortably remain under the 1TB free threshold.
- **Railway (~\$20/mo):** Utilizing usage-based pricing, running a lightweight ASP.NET Core API and a concurrent background ETL worker in a continuous container will cost roughly \$15-\$25

Provider Stack	Compute (API & ETL)	Database Tier	Complexity	Cost/Mo
Vercel / Railway / Supabase	Railway (Containers)	Supabase Pro (Postgres)	Low	~\$65-\$85
AWS	ECS Fargate / App Runner	RDS PostgreSQL (t4g)	High	~\$90-\$120
Microsoft Azure	Azure Container Apps	Azure Database for PG	Medium	~\$70-\$90
Cloudflare	Cloudflare Workers	Cloudflare D1 (SQLite)	High	~\$5-\$25

Table 1: Comparison of Deployment Topologies for a .NET/Angular B2B Application

depending on CPU spikes during the nightly Meta/Google data ingestion.

- **Supabase (\$25/mo):** The Pro tier provides a dedicated PostgreSQL instance, 8GB of database space, daily automated backups, and out-of-the-box Row-Level Security (RLS) configurations.
- **Verdict:** This stack offers the highest Developer Experience (DX) with zero DevOps overhead, making it ideal for a fast-moving, small engineering team.

3.2 Topology 2: Amazon Web Services (AWS)

Stack: Amplify (Frontend) + App Runner/ECS (Backend) + RDS (Database).

Estimated Cost: ~\$90 - \$120/month.

- **Architecture:** AWS provides enterprise-grade scalability. The Angular app is hosted via AWS Amplify. The .NET API and background ETL jobs run on ECS Fargate or App Runner. Data is stored in AWS RDS.
- **Pros:** Industry-standard compliance, infinite scalability, and all resources reside inside a single Virtual Private Cloud (VPC), minimizing latency.
- **Cons:** Extremely high setup complexity. Requires managing IAM roles, VPC networking, NAT Gateways, and writing Terraform infrastructure-as-code scripts. RDS instances have a higher base-line cost for managed Postgres.

3.3 Topology 3: Microsoft Azure

Stack: Static Web Apps (Frontend) + Container Apps (Backend) + Azure Postgres.

Estimated Cost: ~\$70 - \$90/month.

- **Architecture:** As the creators of C# and .NET, Azure offers seamless native integrations. The Angular frontend sits on Azure Static Web Apps, while the API runs in Azure Container Apps (a serverless Kubernetes abstraction).
- **Pros:** Best-in-class integration with ASP.NET Core. Container Apps scale to zero, meaning you only pay for the exact compute used during the nightly ETL bursts.
- **Cons:** The Azure portal is notoriously complex to navigate compared to Vercel and Railway, and

configuring managed databases still requires substantial networking knowledge.

3.4 Topology 4: Cloudflare (The Edge Architecture)

Stack: Cloudflare Pages (Frontend) + Workers (Backend) + Hyperdrive/External DB.

Estimated Cost: ~\$5 - \$25/month.

- **Architecture:** The application is pushed directly to the network edge. Static assets are served via Pages, and API logic runs globally on Cloudflare Workers.
- **Pros:** Unmatched performance (sub-50ms latency globally) and aggressively low pricing. Cloudflare bandwidth is virtually free.
- **Cons: Architectural Incompatibility.** Cloudflare Workers use V8 isolates, meaning you cannot run a standard ASP.NET Core application or Entity Framework Core natively. Adopting this topology would require abandoning C# in favor of TypeScript for the backend, and writing custom cron triggers for the ETL jobs.

3.5 Conclusion

While Cloudflare offers the lowest theoretical cost, it breaks the required ASP.NET Core architecture. Conversely, AWS and Azure offer enterprise scalability but require a dedicated DevOps engineer to configure and maintain. The **Vercel + Railway + Supabase** topology strikes the optimal balance: it keeps fixed costs low (~\$65/month), natively supports containerized C# workflows, handles the persistent Postgres state safely, and automates CI/CD—allowing the team to focus entirely on feature development rather than infrastructure management.

3.5.1 Presentation Layer: Vercel

The Angular frontend application is deployed to Vercel’s global Content Delivery Network (CDN):

- **Resource Allocation:** The Hobby/Pro tiers provide ample monthly bandwidth and Edge Function executions at minimal cost.
- **Capacity Planning:** Serving static assets and client-side routing for an internal group of 30 active concurrent users consumes negligible bandwidth, operating well within base-tier thresholds.

3.5.2 Application Processing Layer: Railway (ASP.NET Core API)

The backend business logic, RBAC, and nightly data pipelines are containerized using Docker and deployed to Railway:

- **Execution Model:** The application runs inside lightweight Linux containers executing the compiled ASP.NET Core API alongside background worker processes (cron jobs) for nightly data ingestion.
- **Cost Structure:** Usage-based pricing ensures that compute credits are primarily utilized during the heavy nightly batch processing windows, maintaining cost-efficiency during standard working hours.

3.5.3 Data Persistence Layer: Supabase (Managed PostgreSQL)

Data storage and analytical query processing are offloaded to a managed PostgreSQL cluster hosted on Supabase:

- **Tenant Isolation:** Supabase native Row-Level Security ensures that consultants only retrieve advertising data for workspaces they have explicitly been granted access to by administrators.
- **Operational Scaling:** While the MVP (5 workspaces) will comfortably fit on lower tiers, accommodating 50-100 workspaces with multi-year daily ad breakdowns per campaign will require scaling storage and compute dynamically as historical data accumulates.

4 Deployment Strategy and CI/CD

4.1 Continuous Integration and Continuous Deployment (CI/CD)

A robust CI/CD pipeline is critical for accelerating iteration cycles safely. To ensure the frontend and backend remain perfectly synchronized during updates—and to prevent broken interfaces resulting from mismatched APIs—the pipeline utilizes a **Blue/Green Deployment (Staging Promotion)** strategy.

Code changes are first pushed to a staging environment where isolated builds of the frontend, backend, and a temporary database branch are deployed concurrently. Once validated via automated tests or manual review, these staging builds are merged into the main branch and atomically promoted to production, effectively swapping the live applications without downtime.

4.1.1 Frontend CI/CD via Vercel

Vercel integrates seamlessly with GitHub to eliminate manual deployment scripts and support atomic promotions.

- When a pull request is created, Vercel automatically generates a unique preview deployment URL, pointing directly to the staging backend API for accurate integration testing.
- Merging the pull request into the main production branch triggers an instant promotion of the validated build, ensuring the UI remains perfectly synced with the incoming backend logic.

4.1.2 Application CI/CD via Railway

Railway provides an integrated CI/CD platform that monitors connected GitHub repositories for changes and orchestrates container lifecycle management.

- Pushing commits to a staging branch prompts Railway to build a new, isolated container in the background, keeping background ingestion jobs and API endpoints up to date without impacting production.
- To ensure code integrity, Railway features a "Wait for CI" configuration that halts deployment until external GitHub Actions (such as automated testing suites for the ETL pipelines) complete successfully.

4.1.3 Data Tier CI/CD via Supabase

Managing stateful database changes during atomic deployments requires strict schema synchronization. Supabase offers a dedicated GitHub integration and native Database Branching to handle migrations safely.

- The Supabase integration watches all pull requests, automatically creating a temporary, isolated database branch containing a replica of the schema for the staging environment to interact with.
- Merging to the main branch safely applies the tested database migrations to the production database immediately before the Vercel and Railway production swaps occur, ensuring new schemas (e.g., adding TikTok API structures) are ready for the incoming backend code.

4.1.4 Automated Testing and CI via GitHub Actions

While Vercel and Railway handle Continuous Deployment (CD), GitHub Actions orchestrate the Continuous Integration (CI) pipeline to guarantee code quality and prevent regressions before any deployment is authorized.

- **Backend Validation (The Gatekeeper):** A dedicated C# workflow restores the .NET 8 SDK, compiles the API, and executes the unit testing suite on every pull request. Railway's "Wait for CI" setting strictly blocks deployment to staging or production if any test fails.
- **Frontend Integrity:** An Angular-specific workflow executes `ng lint` and `ng test` to validate TypeScript strictness, component bindings, and reactive state logic prior to Vercel initiating its build process.
- **Schema Synchronization:** To prevent schema drift between the database and application code, an automated action utilizes the Supabase CLI to regenerate C# models and TypeScript interfaces whenever PostgreSQL migrations are merged, guaranteeing end-to-end type safety.

5 Security and Access Control

Handling sensitive financial and performance data across multiple client accounts requires a strict multi-tenant security model. To prevent cross-workspace data exposure, the system implements a dual-layer authorization strategy combining application-level middleware with database-level security policies.

5.1 Application-Level Authorization

Once the backend receives an HTTP request, the ASP.NET Core API acts as the primary gatekeeper, validating the incoming JSON Web Token (JWT).

- **Role-Based Access Control (RBAC):** Using standard .NET middleware (e.g., `[Authorize(Roles = "Admin")]`), the API restricts sensitive administrative actions. For instance, only users with an Admin role are permitted to execute endpoints that assign or revoke a consultant's access to a specific workspace.
- **Workspace Validation:** For standard data requests, such as fetching KPI charts for a dashboard, the API reads the JWT to verify that the requested `WorkspaceID` exists within the consultant's

array of allowed workspaces. If authorization fails, the API immediately rejects the request with a 403 Forbidden status, preventing unauthorized logic execution.

5.2 Data-Level Authorization (Defense in Depth)

Relying solely on the application layer introduces a single point of failure. If a developer inadvertently omits a tenant filter in an Entity Framework LINQ query, the application could expose all clients' data. To mitigate this risk, the architecture employs a defense-in-depth approach.

5.3 The Transport Layer handles encryption (HTTPS)

Whenever a user logs in, their password and email are sent from the Angular frontend to our ASP.NET Core backend over HTTPS (TLS/SSL). This means the actual network traffic is fully encrypted in transit. Even if someone intercepts the packet on the Wi-Fi, they cannot read it. Therefore, the login payload does not need to be encrypted before sending.

5.4 Cryptographic Controls

To further secure the application and support defense-in-depth, cryptographic measures are applied across data in transit and at rest.

- **Data in Transit (HTTPS):** Whenever a user logs in, their password and email are sent from the Angular frontend to the ASP.NET Core backend over HTTPS (TLS/SSL). The network traffic is fully encrypted in transit, neutralizing packet interception risks.
- **Data at Rest (API Secrets):** API secrets are never saved as plain text in PostgreSQL. Standard .NET cryptographic libraries (AES-256) inside the C# backend encrypt the secret strings prior to persisting them to Supabase. They are decrypted only in memory when the nightly worker fetches ad data.
- **Password Hashing:** Plain-text login passwords are not stored. The system utilizes the modern standard built into ASP.NET Core: `PasswordHasher<TUser>`, which implements PBKDF2 with HMAC-SHA256. It automatically handles "salting" to ensure identical passwords produce completely unique hashes.

5.5 Compliance and ISO/IEC 27001 Alignment

Enterprise clients frequently require assurance that their financial and marketing data is handled according to international security standards, most notably ISO/IEC 27001. While compliance is an organizational certification, the system architecture is designed to enforce the necessary technical controls to support an Information Security Management System (ISMS).

- **The Shared Responsibility Model:** By utilizing managed cloud providers (Vercel, Supabase, and Railway), the infrastructure layer automatically inherits robust physical and network security compliance. Supabase and Vercel maintain ISO/IEC 27001 certifications, while Railway holds SOC 2 Type 2 compliance (a widely accepted enterprise equivalent). Because the cloud vendors handle

infrastructure hardening, the engineering focus shifts entirely to application-level security and data governance.

- **Comprehensive Audit Logging:** To fulfill ISO requirements for non-repudiation and traceability, the ASP.NET Core backend implements an audit trail. Critical actions—such as modifying third-party API keys, exporting client data, or altering workspace permissions—are logged in the database with strict timestamps and the associated user UUID.
- **Cryptographic Controls and Access:** The architecture strictly enforces data-in-transit encryption via forced TLS/HTTPS across all endpoints. At rest, sensitive credentials (e.g., Meta and Google API tokens) are secured using AES-256 encryption via the .NET cryptographic libraries before being persisted to the PostgreSQL database.
- **Identity and Access Management (IAM):** The system adheres to the Principle of Least Privilege. The JWT-driven Role-Based Access Control (RBAC) ensures that consultants cannot access workspaces unless explicitly mapped in the database, preventing unauthorized lateral movement across client accounts.

5.6 Risk Management Framework (ISO 31000 Alignment)

Beyond strict security compliance, the system architecture aligns with the principles of ISO 31000 to proactively identify, assess, and mitigate operational and infrastructure risks that could impact business continuity for PL & Partners.

- **Third-Party Dependency Mitigation:** The platform relies heavily on external advertising APIs (Meta, Google, TikTok). To prevent external API outages or rate limits from cascading into frontend downtime, the architecture decouples data ingestion via a background ETL worker. This ensures the Angular frontend remains highly responsive, serving the most recently successfully synced database records even if an external provider experiences an outage.
- **Infrastructure Portability:** To mitigate vendor lock-in and systemic cloud provider failure, the ASP.NET Core backend utilizes standard Docker containerization. While currently deployed to Railway for developmental velocity, the containerized nature of the API ensures it can be rapidly migrated to alternative environments, such as AWS Fargate or Azure Container Apps, if required.
- **Data Preservation and Error Recovery:** To protect against catastrophic human error, the application enforces "Soft Deletes" across all critical database tables. This architectural rule, paired with the automated daily backups and Point-in-Time Recovery (PITR) provided by Supabase, ensures that accidental workspace deletions can be reversed with zero permanent data loss.

6 System Architecture Topology

The diagram below (Figure 1) illustrates the end-to-end data flow of the proposed solution. It maps out how the Angular frontend communicates with the ASP.NET Core backend via secure JWTs, and how the background worker independently orchestrates data ingestion from third-party advertising APIs (Meta, Google, TikTok) into the securely isolated PostgreSQL database.

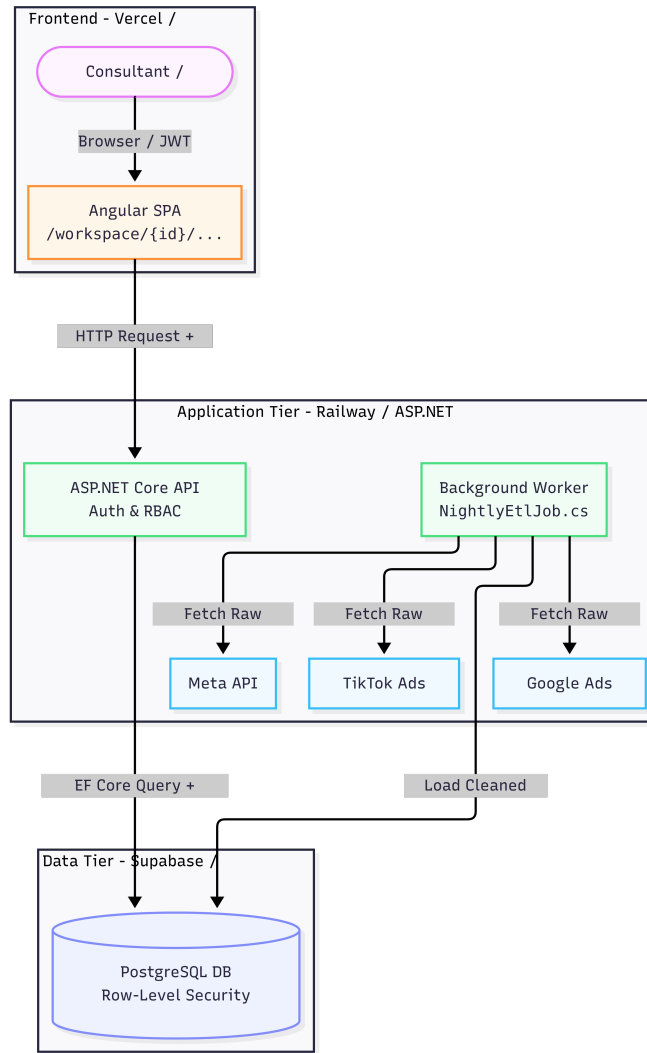


Figure 1: High-level system architecture detailing the client, application, and data tiers, along with the nightly ETL data ingestion workflow.

7 Frontend Architecture and Navigation

To support a seamless user journey and strict tenant isolation, the Angular frontend routing is logically divided into global navigation and workspace-specific contexts.

7.1 Global Pages (Cross-Client Navigation)

These routes handle authentication, access management, and high-level navigation before a consultant enters a specific client's data environment.

- **Login Page:** The secure entry point for authentication, generating the JSON Web Token (JWT) containing the user's role and workspace access claims.
- **Workspace Directory (Landing Page):** Upon authenticating, consultants are presented with a directory of available client workspaces. This view is dynamically filtered by the backend so consultants only see the specific clients they are explicitly authorized to access.
- **Admin Control Panel (Restricted):** An administrative interface for access management. Ad-

mins use this page to provision new client workspaces, invite new consultants, and map which consultants have access to which clients.

7.2 Workspace-Specific Pages (Tenant Context)

Selecting a client from the directory routes the user into that specific workspace. The URL updates to include the `WorkspaceID` (e.g., `/workspace/{id}/dashboard`), establishing the context for the persistent left-hand menu. This ID is subsequently passed to the ASP.NET Core API to enforce data-level authorization on all data fetching.

- **Dashboards (Overview)** (`/workspace/{id}/dashboard`): The primary workspace landing page featuring high-level KPI charts, aggregated tables across Meta, Google, and TikTok, and top-line spending summaries.
- **Budget & Pacing** (`/workspace/{id}/budget`): A dedicated financial view tracking the client's ad spend pacing against predefined monthly, quarterly, and yearly budget targets.
- **Analyses & Tools** (`/workspace/{id}/analyses`): An advanced analytical view for deep-dive reporting, including performance monitoring, diminishing return modeling, and historical ad comparisons.
- **Data Connections (Admin Only)** (`/workspace/{id}/connections`): A configuration interface where API keys and credentials for third-party advertising platforms are input and managed for the nightly ETL pipelines.

8 Implementation and Local Tooling

8.1 Local Development Environment

To ensure a streamlined and efficient developer workflow across the entire stack, the local environment relies on a unified tooling strategy optimized for low overhead and cross-platform compatibility.

- **Unified Editor (Visual Studio Code)**: VS Code serves as the single Integrated Development Environment (IDE) for both the Angular (TypeScript) frontend and the ASP.NET Core (C#) backend. By utilizing the official C# Dev Kit and the Angular Language Service extensions, the environment provides native type-checking, seamless code navigation, and integrated debugging across both languages without the memory overhead of running multiple heavyweight IDEs.
- **Database Emulation (Supabase CLI)**: Rather than relying on a remote cloud connection during active coding, the Supabase CLI is utilized to orchestrate a localized PostgreSQL instance via Docker. This grants access to a local instance of Supabase Studio, allowing for the safe offline testing of database migrations, schema designs, and Row-Level Security (RLS) policies before committing changes to the staging environment.

8.2 Project Directory Structure

To maintain a scalable codebase as the platform grows from an MVP to supporting over 100 client workspaces, both the frontend and backend are organized using a Feature-Based Architecture. This approach groups files by their business domain rather than their technical type, minimizing cognitive load for developers.

8.2.1 Frontend (Angular)

The Angular workspace utilizes modern Standalone Components, dividing the application into core infrastructure, shared UI elements, and distinct feature pages.

```
pl-partners-frontend/  
src/  
  app/  
    core/           # Singleton services & app-wide config  
      auth/         # JWT interceptors and Auth guards  
      api.service.ts # Base HTTP client configuration  
    shared/         # Reusable UI components  
      charts/       # KPI charts and data visualizations  
      layout/       # Persistent left-hand menu and header  
    features/       # Route-based feature modules  
      login/        # Authentication page  
      directory/    # Global workspace selection grid  
      workspace/    # Tenant-specific pages (requires ID)  
        dashboard/  
        budget/  
        analyses/  
        connections/  
      app.routes.ts # Global routing and tenant parameters  
      app.config.ts # Application bootstrapping  
    environments/   # Staging vs. Production URL configs
```

8.2.2 Backend (ASP.NET Core)

The C# backend separates HTTP routing, business logic, background processing, and database interactions into clear layers to support the nightly ETL pipeline safely.

```
PLPartners.Backend/  
  Controllers/      # HTTP endpoints (The Presentation Layer)  
    AuthController.cs # Login and JWT generation  
    WorkspaceController.cs # RBAC and tenant validation  
    AnalyticsController.cs # Dashboard data endpoints  
  Core/             # Business Logic & Interfaces  
    Models/         # Entity classes (User, Workspace, Metric)
```

DTOs/	# Data Transfer Objects for API payloads
Interfaces/	# Contracts for services and repositories
Infrastructure/	# External dependencies
Database/	# EF Core ApplicationDbContext and Migrations
Integrations/	# Meta, Google, and TikTok API clients
Workers/	# Background Services
NightlyEtlJob.cs	# Scheduled task for batch data ingestion
appsettings.json	# Environment variables (DB strings, JWT keys)
Program.cs	# Dependency injection and middleware setup

8.3 Database Schema and Entity Relationships

To support the multi-tenant architecture and facilitate the nightly batch ETL process, the PostgreSQL database is structured around five core tables. The schema utilizes UUIDs to prevent enumeration attacks and relies on strict Foreign Key constraints to maintain relational integrity.

- **Users:** Stores consultant accounts and their global administrative roles.
 - Id (UUID, Primary Key)
 - Email (String, Unique)
 - GlobalRole (Enum: Admin, Consultant)
- **Workspaces:** Represents the individual client environments.
 - Id (UUID, Primary Key)
 - ClientName (String)
 - CreatedAt (Timestamp)
- **WorkspaceAccess (Many-to-Many):** The junction table that acts as the source of truth for Role-Based Access Control, mapping which consultants have access to which client workspaces.
 - UserId (UUID, Foreign Key → Users.Id)
 - WorkspaceId (UUID, Foreign Key → Workspaces.Id)
 - *Note: Utilizes a Composite Primary Key on (UserId, WorkspaceId).*
- **ApiConnections:** Stores the configuration needed for the nightly background jobs to reach out to third-party platforms.
 - Id (UUID, Primary Key)
 - WorkspaceId (UUID, Foreign Key → Workspaces.Id)
 - Platform (Enum: Meta, Google, TikTok)
 - EncryptedApiKey (String)
 - ExternalAccountId (String)
- **AdMetrics (The Data Warehouse Table):** The central repository for all ingested advertising data, optimized for read-heavy dashboard queries.
 - Id (UUID, Primary Key)
 - WorkspaceId (UUID, Foreign Key → Workspaces.Id) [*Row-Level Security Enforced*]
 - Platform (Enum: Meta, Google, TikTok)

- RecordDate (Date)
- CampaignName, AdName (String)
- Spend, Impressions, Clicks, Conversions (Numeric)